Mathlingua Cheat Sheet

High-Level Language Syntax Reference

A concise, high-level map of Mathlingua syntax. For structural constructs, only the first section name is shown unless disambiguation is required. For in-depth specifications and worked examples, see `docs/language.md`, `docs/structural_syntax.md`, and `docs/formulation_syntax.md`.

Syntax Name	Syntax Example	Description
Top-Level Structural Groups		
`Title`	`Title: "..."`	Top-level document title
`SectionTitle`	`SectionTitle: "..."`	Major section heading within a document
`SubsectionTitle`	`SubsectionTitle: "..."`	Subsection heading within a section
`Text`	`Text: "..."`	Standalone narrative prose block
`Writing`	`Writing: "name :~> body"`	Custom LaTeX rendering alias for a name or symbol
`Declares`	`Declares: f(x__)`	Declares a mathematical symbol, function shape, or operator
`Defines`	`Defines: X := \set@{...}`	Defines a mathematical concept or object with explicit construction
`Realizes`	`Realizes: Nb := \naturals`	Supplies concrete implementation for an abstract definition
`Refines`	`Refines: x is \real`	Specializes a declaration with additional constraints or properties
`States`	`States:`	Declares a named mathematical property, condition, or theorem-like claim
`Axiom`	`Axiom:`	Assumed foundational mathematical truth
`Theorem`	`Theorem:`	Proven mathematical statement (optionally with proof)
`Conjecture`	`Conjecture:`	Unproven mathematical hypothesis
`Example`	`Example:`	Concrete instance or demonstration of a concept
`Equivalent`	`Equivalent:`	States mutual equivalence between multiple mathematical claims
`Relation` (undirected)	`Relation: ... between: ... and: ...`	Symmetric or undirected relationship between entities
`Relation` (directed)	`Relation: ... from: ... to: ...`	Directed relationship from a source to a target entity
`Disambiguates`	`Disambiguates:`	Dispatches overloaded operators or functions by argument types
`Specify`	`Specify:`	Collection-wide numeric literal type mapping (decimal, int, etc.)
`Person`	`Person: "Name"`	Author, contributor, or mathematician metadata
`Resource`	`Resource:`	Bibliographic source (book, paper, website) cited by entries
`Topic`	`Topic: "Name"`	Subject tag or domain classification
`TextTheorem`	`TextTheorem: "..."`	Opaque prose placeholder for an unformalized theorem
`TextAxiom`	`TextAxiom: "..."`	Opaque prose placeholder for an unformalized axiom
`TextConjecture`	`TextConjecture: "..."`	Opaque prose placeholder for an unformalized conjecture
`TextDefinition`	`TextDefinition: "..."`	Opaque prose placeholder for an unformalized definition
Structural Clauses		
`forAll`	`forAll: x_`	Universal quantifier over variables or specifications
`exists`	`exists: x_`	Existential quantifier asserting at least one satisfying witness

Continued on next page...

Syntax Name	Syntax Example	Description
`existsUnique`	`existsUnique: x_`	Uniqueness quantifier asserting exactly one satisfying witness
`if`	`if: P`	Conditional premise implying following clauses
`have` (iff)	`have: P ... iff: Q`	Biconditional assertion stating equivalence between facts
`have` (asserting)	`have: P ... asserting: Q`	Deductive step establishing claim with local assertions and justifications
`piecewise`	`piecewise:`	Branching conditional definition or statement (`if/then/elseIf/else`)
`let`	`let: x_`	Introduces locally bound variables or hypotheses
`not`	`not: P`	Logical negation of a nested clause
`allOf`	`allOf:`	Conjunction requiring all nested clauses to hold
`anyOf`	`anyOf:`	Disjunction requiring at least one nested clause to hold
`oneOf`	`oneOf:`	Exclusive disjunction requiring exactly one nested clause to hold
`equivalently`	`equivalently:`	Asserts that all nested clauses are mutually equivalent
Supporting Sections		
`using:`	`using: x is \set`	Fixed parameters or contextual variables in scope
`when:`	`when: x != 0`	Preconditions or validity conditions for a definition or rule
`where:`	`where: x "in" X`	Conditions constraining quantified variables
`given:`	`given: x is \real`	Arbitrary given objects or hypotheses in a theorem, axiom, or conjecture
`then:`	`then: P`	Conclusion or consequence of premises
`iff:`	`iff: P`	Necessary and sufficient condition
`satisfies:`	`satisfies: x > 0`	Invariant or property guaranteed by a declaration or refinement
`extends:`	`extends: X is \group`	Declares inheritance or specialization of base types or structures
`abstractly:`	`abstractly:`	Marks a definition specification as abstract, requiring realization
`expresses:`	`expresses: P`	Logical expression that a definition or realization equates to
`Requires:`	`Requires:`	Prerequisites (capabilities or definitions) needed to use an entry
`Enables:`	`Enables:`	Capabilities, views, or conversions provided by an entry
`Documented:`	`Documented:`	Human-readable documentation (`called:`, `written:`, `description:`, etc.)
`Justification:`	`Justification:`	Deductive steps discharging formulation labels (`[:label:]`)
`Aliases:`	`Aliases:`	Alternative notation aliases (`:=>`, `:->`, `:<->:`)
`References:`	`References:`	Citations to external resources (`$resource` or `. [label]: $resource`)
`Proof:`	`Proof: "..."`	Informal or formal proof body for a theorem
`Metadata:`	`Metadata:`	Versioning and tracking metadata (`id:`, `version:`)
`Id:`	`Id: "uuid"`	Unique UUID identifier for any top-level item
Headings and Headers		
Command Heading	`[\function:on{A}:to{B}]`	Identifies group by full command pattern with inputs
Operator Heading	`[a \:(nonempty)::subset:/ b]`	Identifies group by infix operator or spec-infix pattern
Label Heading	`[label]`	Local label naming a clause, step, or justification

Continued on next page...

Syntax Name	Syntax Example	Description
Author Header	`[@author.id]`	Identifies a person entry or cites an author
Resource Header	`[$resource.id]`	Identifies a bibliographic resource or cites a reference
Topic Header	`[#topic.id]`	Identifies a topic classification entry
Formulations, Declarations, and Operators		
Structural Assignment	`X ::= {x_ : ...}`	Defines a structural pattern or alias matching literal forms
Value Assignment	`x := 0`	Binds a symbol or variable to a specific value or construction
Equality	`x = y`	Asserts mathematical equality between two expressions
Inequality	`x != y`	Asserts mathematical inequality between two expressions
Type Assertion	`x is \real`	Asserts that an expression belongs to a type or satisfies a specification
Type Query	`x is? \real`	Boolean predicate testing whether an expression satisfies a type
Refined Type / View	`x is \group via X`	Type assertion through an explicit view or structural adapter
Spec Literal	`_ is \real`	Anonymous specification value (`\\specification`)
Quoted Spec	`x "in" X`	Binary relation or membership specification
Quoted Spec Query	`x "in"? X`	Boolean predicate testing a quoted binary relation
Infix Command	`a \.plus./ b`	Infix invocation of a defined command
Infix Spec	`a \:subset:/ b`	Infix specification statement
Infix Spec Query	`a \:subset?:/ b`	Boolean predicate testing an infix specification
Adjective Refinement	`\(nonempty)::set`	Modifies a command type with a refined adjective constraint
Build / Cast	`\set@{...}`	Applies a command type constructor or cast to a raw value
Function Literal	`(x_) => x_ + 1`	Anonymous lambda/mapping literal from inputs to an expression
Function Type	`(_ is \real) -> (_ is \real)`	Type signature for functions mapping domain to codomain
Expression Alias	`f(x_) :=> x + x`	Expression or operator alias mapping a form to an expression reduction
Directed Spec Alias	`x_ "in" R :-> x is \real`	Directed specification capability asserting that the LHS implies the RHS
Equivalence Spec Alias	`x_ "in" S :<->: x is \integer`	Bidirectional specification capability asserting LHS and RHS equivalence
Named Infix Operator	`a |*| b`	Infix named operator invocation (also `a :|*| b`, `a |*|: b`, `a :|*|: b`)
Member Access / Call	`M.*` / `M.*(x, y)`	Accesses or invokes a field/operation of a structured object
Set Literal / Builder	`{x_ : x_ "in" X}`	Set comprehension or collection literal
Tuple	`(a, b)`	Ordered sequence of two or more elements
Variadic / Slice	`x...` / `x[1...n]`	Represents an arbitrary sequence of arguments or array slice
Inferred Symbol	`x?` / `x?_`	Parameter whose value is inferred from context
Justification Label	`(x + y)[:label:]` / `[:l1, l2:]`	Tags formulation to be proved in `Justification:` (supports multiple labels)
Formulation Citation	`(x + y)(:res:)`	Cites a bibliographic resource directly on a formulation

Continued on next page...

Syntax Name	Syntax Example	Description
Line Label	`. [label]: x + y`	Labels a single-line formulation or text argument in a section
Builtin Types and Commands		
Builtin Types	`\\type`, `\\statement`, `\\expression`, `\\specification`, `\\anything`	Language primitive types for classification and constraints
Builtin Capability Target	`\\abstract`	Placeholder target for abstract specification capabilities in `:->` or `:<->:`
Builtin Connectives	`\\not{P}`, `\\and{...}`, `\\or{...}`, `\\oneOf{...}`	Formulation-level logical connectives (also `\\allOf`, `\\anyOf`)
Builtin Quantifiers	`\\forAll{...}:then{...}`, `\\exists{...}`, `\\let{...}`	Formulation-level quantifiers (`\\forAll`/`\\forall`, `\\exists`, `\\existsUnique`, `\\let`)
Builtin Conditionals	`\\if{...}:then{...}`, `\\have{...}:iff{...}`, `\\piecewise{}`	Formulation-level conditional statements (`\\if`, `\\have`, `\\piecewise`)
Builtin Variadic Commands	`\\map{...}:to{...}`, `\\leftReduce{...}:on{...}`	Pointwise slice mapping (`\\map`) and reductions (`\\leftReduce`, `\\rightReduce`)
Prose Delimiters and Citations		
Inline Formulation	`{. x + y .}`	Embeds an inline math expression inside prose text
Display Formulation	`{{. x + y .}}`	Embeds a centered display math expression inside prose text
Theorem Reference	`{: \thm :}`	Links to a theorem, axiom, or conjecture within prose
Prose Citation	`''quote''(:ref:)`	Cites a bibliographic resource with an optional quoted phrase
Scoped Context	`<<scope>> ... <</scope>>`	Constrains mathematical context and notations within prose
Command-Line Interface (CLI)		
Check	`mlg check`	Type-checks and validates all `.mlg` files in the collection
Search	`mlg search <query>`	Searches collection for matching symbols, commands, or text
Structure	`mlg structure`	Validates and visualizes collection document hierarchy and links
Format	`mlg format`	Formats `.mlg` files into canonical indentation and style
View	`mlg view`	Launches local web viewer server to browse knowledge base
Export	`mlg export`	Exports collection to static HTML website
Clean	`mlg clean`	Removes build artifacts and cached output files
Init	`mlg init`	Initializes a new Mathlingua collection in current directory
Release	`mlg release`	Creates a distribution bundle or archive of collection
LSP	`mlg lsp`	Starts Language Server Protocol server for IDE integration
Version	`mlg version`	Prints installed Mathlingua version information